

M2M OAuth2 Token Issuance over mTLS — AWS Implementation Guide

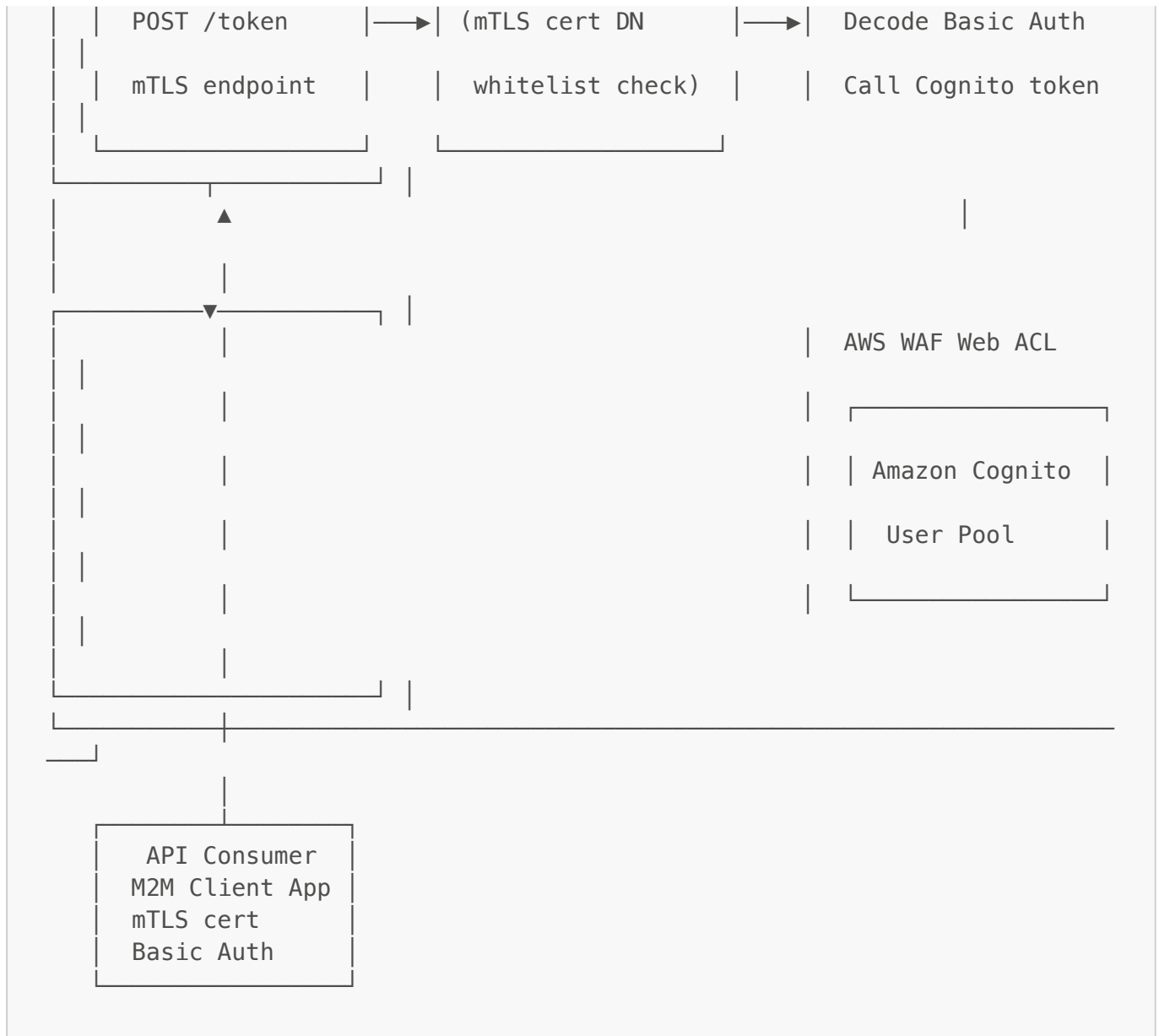
Table of Contents

1. [Architecture Overview](#)
2. [Flow Description](#)
3. [Security Layers](#)
4. [Prerequisites](#)
5. [Repository Structure](#)
6. [Terraform Infrastructure](#)
 - [main.tf](#)
 - [variables.tf](#)
 - [locals.tf](#)
 - [ssm.tf](#)
 - [cognito.tf](#)
 - [waf.tf](#)
 - [lambda.tf](#)
 - [api_gateway.tf](#)
 - [outputs.tf](#)
7. [Lambda Function Code](#)
 - [Token Proxy](#)
 - [Lambda Authorizer](#)
 - [IP Range Updater](#)
8. [WAF Rules Reference](#)
9. [Deployment Steps](#)
10. [Post-Deployment Verification](#)
11. [Operational Notes](#)

1. Architecture Overview

This solution enables a **Machine-to-Machine (M2M) client** to obtain an OAuth2 access token from **Amazon Cognito User Pool** through a secure, multi-layered proxy. Direct access to the Cognito token endpoint is fully blocked — all requests must flow through AWS API Gateway.





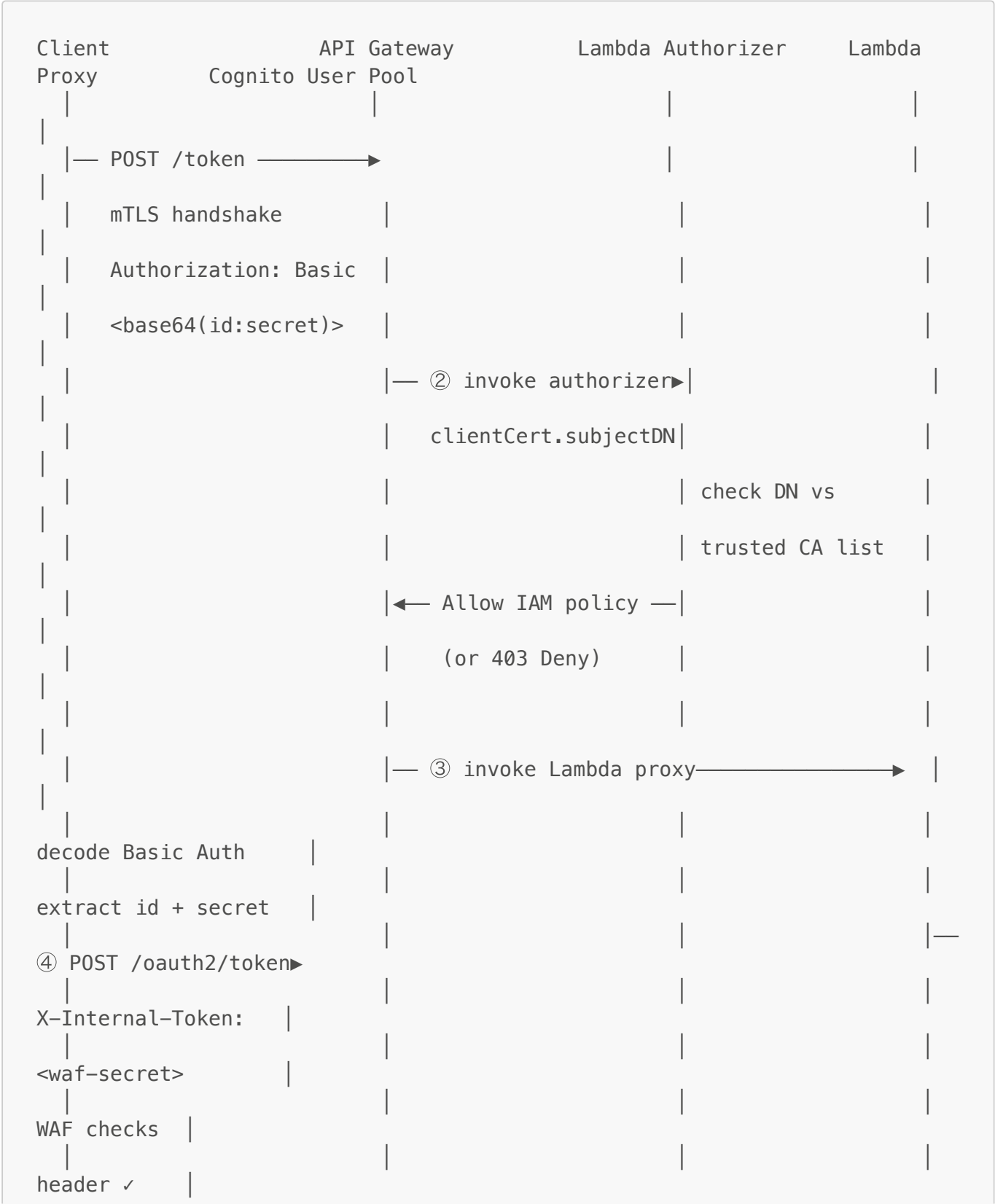
AWS Services used:

Service	Role
API Gateway (REST)	mTLS termination, request routing
Lambda — Authorizer	REQUEST-type authorizer; validates client certificate DN
Lambda — Token Proxy	Decodes Basic Auth, injects WAF secret header, proxies to Cognito
Lambda — IP Range Updater	Keeps WAF IP set current when AWS publishes new ranges
Amazon Cognito User Pool	Issues OAuth2 <code>client_credentials</code> access tokens
AWS WAF	Web ACL on Cognito; blocks all requests except from the Lambda proxy
AWS SSM Parameter Store	Stores the WAF internal secret (SecureString)
Amazon S3	Hosts the mTLS truststore PEM (trusted CA certificates)

Service	Role
Amazon SNS	Triggers IP range updater when AWS publishes new IP ranges
AWS CloudWatch	Logs for API Gateway, Lambda functions, WAF

2. Flow Description

Step-by-step request flow





Steps explained

Step	Actor	Action
①	Client	Opens mTLS connection presenting its client certificate; sends <code>Authorization: Basic base64(clientId:clientSecret)</code>
②	API Gateway → Lambda Authorizer	Extracts <code>requestContext.identity.clientCert.subjectDN</code> ; checks DN against trusted CA whitelist; returns IAM Allow or Deny policy
③	API Gateway → Lambda Proxy	Forwards the authorized POST request with all headers to the backend Lambda
④	Lambda Proxy → Cognito	Decodes Basic Auth header; calls <code>POST /oauth2/token</code> with <code>grant_type=client_credentials</code> ; injects <code>X-Internal-Token: <secret></code>
⑤	Cognito → Lambda Proxy	Returns <code>{ access_token, token_type, expires_in }</code>
⑥	Lambda Proxy → Client	Returns Cognito token response unchanged to the API consumer

3. Security Layers

Why not allow direct client access to Cognito?

Cognito's `/oauth2/token` endpoint is public on the internet. Without additional controls, any client with valid `client_id` / `client_secret` credentials can obtain a token from anywhere. This solution adds three layers of defense in front of Cognito:

```

Layer 1 (Transport)  - mTLS at API Gateway edge
Layer 2 (Identity)   - Lambda Authorizer validates certificate DN against
                      trusted CA whitelist
Layer 3 (Network)    - WAF Web ACL on Cognito blocks everything except
                      requests with the internal secret header
  
```

WAF Rule Priority Table

Priority	Rule	Default Action	Purpose
1	AWS Managed Common Rule Set	Count/Block	OWASP baseline (SQLi, XSS, bad inputs)
2	Block non <code>/oauth2/token</code> paths	Block	Restrict to token endpoint only
3	Require <code>X-Internal-Token</code> header	Allow	Primary gate — only Lambda proxy knows the secret
4	Allow AWS AMAZON IP ranges	Allow	Secondary belt-and-suspenders; source must be AWS-owned IPs
5	Rate limit 100 req / 5 min / IP	Block	Brute-force and credential stuffing protection
—	Default action	Block 403	Everything not explicitly allowed is denied

Why use a secret header and not just IP ranges?

API Gateway **does not have static, predictable public IPs**. The AWS AMAZON IP ranges cover all AWS-owned space in a region — too broad to be the sole control. The `X-Internal-Token` header is only known to the Lambda proxy, making it cryptographically enforced regardless of source IP.

Approach	Reliability	Notes
Secret header (<code>X-Internal-Token</code>)	✅ Strong	Proof that request came from <i>your</i> Lambda proxy
AWS IP ranges (AMAZON prefix)	⚠️ Broad	Good secondary control; must be refreshed when AWS updates ranges
Combined (both)	✅✅ Defense-in-depth	Recommended — what this solution implements

4. Prerequisites

Tools

- [Terraform](#) >= 1.5.0
- [AWS CLI](#) >= 2.x configured with appropriate permissions
- Python 3.12 (to build Lambda packages)
- `zip` utility

AWS IAM permissions required for deployment

The IAM principal running `terraform apply` needs permissions for:

```

apigateway:*
cognito-idp:*
lambda:*
wafv2:*
s3:*
ssm:PutParameter, GetParameter, DeleteParameter
iam:CreateRole, AttachRolePolicy, PutRolePolicy, PassRole
logs:CreateLogGroup, PutRetentionPolicy
sns:Subscribe

```

Certificates

Before deploying, prepare:

1. **mTLS truststore** — a PEM file containing the CA certificate(s) whose client certificates will be trusted. Upload to S3 after the bucket is created.
2. **ACM Certificate** — a TLS certificate for the API Gateway custom domain (must be in the same region as the API Gateway).

5. Repository Structure

```

oauthLab/
├── SOLUTION.md                ← this file
├── terraform/
│   ├── main.tf               # Terraform + AWS provider config
│   ├── variables.tf          # All input variables with descriptions
│   ├── locals.tf             # name_prefix and common_tags
│   ├── ssm.tf                # Random secret + SSM SecureString
│   └── cognito.tf            # User Pool, domain, M2M app client,
WAF association
├── waf.tf                    # WAF Web ACL (5 rules), IP set,
CloudWatch logging
├── lambda.tf                 # All Lambda functions + IAM
roles/policies
├── api_gateway.tf            # REST API, mTLS domain, authorizer,
SNS subscription
├── outputs.tf                # Exported resource references
├── terraform.tfvars.example  # Template – copy to terraform.tfvars
├── lambda/
│   ├── token_proxy.py        # Backend Lambda: Basic Auth decode +
Cognito proxy
│   └── authorizer.py         # Authorizer Lambda: cert DN whitelist
check
│   ├── ip_range_updater.py   # Updater Lambda: refreshes WAF IP set
│   └── build.sh              # Packages all three into
token_proxy.zip

```

6. Terraform Infrastructure

main.tf

Declares the Terraform version constraint and the AWS + Random providers.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.5"
    }
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = local.common_tags
  }
}
```

variables.tf

Variable	Type	Default	Description
<code>aws_region</code>	string	<code>"us-east-1"</code>	AWS region
<code>environment</code>	string	<code>"prod"</code>	Deployment environment
<code>project</code>	string	<code>"oauth-m2m"</code>	Project name prefix
<code>cognito_domain_prefix</code>	string	—	Cognito hosted domain prefix (globally unique)
<code>acm_certificate_arn</code>	string	—	ACM cert ARN for API Gateway custom domain
<code>truststore_s3_version</code>	string	<code>null</code>	S3 version ID of the truststore PEM

Variable	Type	Default	Description
<code>lambda_zip_path</code>	string	<code>"../lambda/token_proxy.zip"</code>	Path to the Lambda ZIP package
<code>waf_rate_limit</code>	number	<code>100</code>	Max requests per 5-min window per IP
<code>amazon_ip_ranges</code>	list(string)	See file	Bootstrap AWS AMAZON CIDRs
<code>tags</code>	map(string)	<code>{}</code>	Additional resource tags

locals.tf

```

locals {
  name_prefix = "${var.project}-${var.environment}"

  common_tags = merge(
    {
      Project      = var.project
      Environment  = var.environment
      ManagedBy    = "terraform"
    },
    var.tags
  )
}

```

ssm.tf

- Generates a 64-character random string as the WAF internal header secret.
- Stores it in SSM Parameter Store as a `SecureString` at `/<project>-<env>/waf/internal-header-secret`.
- The Lambda proxy reads this value at runtime to inject into the `X-Internal-Token` header.

cognito.tf

Creates:

- **Cognito User Pool** — admin-only user creation, no self-signup.
- **Cognito Hosted Domain** — exposes the `/oauth2/token` endpoint.
- **M2M App Client** — `client_credentials` grant only, generates a client secret, scoped to the resource server.
- **Resource Server** — defines the custom OAuth2 scope (`read`).
- **WAF Web ACL Association** — links the WAF Web ACL to the User Pool.

waf.tf

Creates the WAF Web ACL with five rules (see [WAF Rules Reference](#)):

- IP Set for AWS AMAZON CIDRs (secondary defense, auto-refreshed by Lambda).
- Full request logging to CloudWatch with `X-Internal-Token` redacted from logs.

lambda.tf

Creates three Lambda functions with dedicated IAM roles:

Function	Handler	Timeout	Purpose
<code>token-proxy</code>	<code>token_proxy.lambda_handler</code>	10s	Decodes Basic Auth, proxies to Cognito
<code>lambda-authorizer</code>	<code>authorizer.lambda_handler</code>	5s	Validates mTLS cert DN whitelist
<code>ip-range-updater</code>	<code>ip_range_updater.lambda_handler</code>	30s	Refreshes WAF IP set from ip-ranges.json

IAM permissions follow least-privilege: each role only gets the specific actions it needs.

api_gateway.tf

Creates:

- **REST API** with a `POST /token` method.
- **REQUEST-type Lambda Authorizer** using `context.identity.clientCert.subjectDN` as the identity source. TTL is set to `0` (no caching — each request's cert is checked fresh).
- **Lambda proxy integration** for the backend Token Proxy function.
- **mTLS custom domain** with an S3 truststore.
- **SNS subscription** to `AmazonIpSpaceChanged` — triggers the IP range updater Lambda when AWS publishes new IP ranges.

outputs.tf

Output	Description
<code>api_gateway_invoke_url</code>	mTLS token endpoint (custom domain)
<code>api_gateway_stage_url</code>	Direct stage URL
<code>cognito_user_pool_id</code>	User Pool ID
<code>cognito_user_pool_arn</code>	User Pool ARN
<code>cognito_m2m_client_id</code>	M2M app client ID
<code>cognito_m2m_client_secret</code>	M2M app client secret (sensitive)
<code>waf_web_acl_arn</code>	WAF Web ACL ARN

Output	Description
<code>waf_ip_set_arn</code>	WAF IP set ARN
<code>truststore_bucket</code>	S3 bucket name for the mTLS truststore
<code>waf_internal_secret_ssm_path</code>	SSM path for the WAF internal header secret

7. Lambda Function Code

Token Proxy (`token_proxy.py`)

Reads the WAF secret from SSM at cold start, decodes the client's Basic Auth header, then calls the Cognito `/oauth2/token` endpoint with the injected `X-Internal-Token` header. Returns the Cognito response payload unchanged.

```
import boto3
import os
import base64
import urllib.request
import urllib.parse
import json

ssm = boto3.client("ssm")

def _get_waf_secret():
    resp = ssm.get_parameter(
        Name=os.environ["WAF_SECRET_SSM_PATH"],
        WithDecryption=True,
    )
    return resp["Parameter"]["Value"]

# Loaded once per cold start
WAF_INTERNAL_SECRET = _get_waf_secret()
COGNITO_DOMAIN = os.environ["COGNITO_DOMAIN"]

def lambda_handler(event, context):
    # 1. Decode Basic Auth header
    auth_header = (event.get("headers") or {}).get("authorization", "")
    if not auth_header.lower().startswith("basic "):
        return _error(401, "invalid_client", "Missing Basic Auth header")

    try:
        decoded = base64.b64decode(auth_header[6:]).decode("utf-8")
        client_id, client_secret = decoded.split(":", 1)
    except Exception:
        return _error(401, "invalid_client", "Malformed Basic Auth header")

    # 2. Build request to Cognito – inject WAF secret header
    body = urllib.parse.urlencode({
        "grant_type": "client_credentials",
```

```

        "client_id"      : client_id,
        "client_secret"  : client_secret,
        "scope"          : (event.get("queryStringParameters") or
    {}).get("scope", ""),
    }).encode("utf-8")

    req = urllib.request.Request(
        url      = f"{COGNITO_DOMAIN}/oauth2/token",
        data     = body,
        method   = "POST",
        headers  = {
            "Content-Type"      : "application/x-www-form-urlencoded",
            "Authorization"     : auth_header,
            "X-Internal-Token" : WAF_INTERNAL_SECRET,    # WAF gate
        },
    )

    try:
        with urllib.request.urlopen(req) as resp:
            token_response = json.loads(resp.read())
    except urllib.error.HTTPError as e:
        error_body = json.loads(e.read())
        return {
            "statusCode": e.code,
            "headers"   : {"Content-Type": "application/json"},
            "body"      : json.dumps(error_body),
        }

    # 3. Return Cognito response as-is
    return {
        "statusCode": 200,
        "headers"   : {"Content-Type": "application/json"},
        "body"      : json.dumps(token_response),
    }

def _error(status, error, description):
    return {
        "statusCode": status,
        "headers"   : {"Content-Type": "application/json"},
        "body"      : json.dumps({
            "error"           : error,
            "error_description": description,
        }),
    }

```

Lambda Authorizer ([authorizer.py](#))

REQUEST-type authorizer. Extracts the client certificate's Subject DN from the API Gateway request context and checks it against the [TRUSTED_CA_DNS](#) environment variable (comma-separated list). Returns an IAM policy allowing or denying the request.

```

import os
import json

TRUSTED_CA_DNS = set(
    dn.strip()
    for dn in os.environ.get("TRUSTED_CA_DNS", "").split(",")
    if dn.strip()
)

def lambda_handler(event, context):
    client_cert = event.get("requestContext", {}).get("identity",
    {}).get("clientCert", {})
    subject_dn = client_cert.get("subjectDN", "")

    method_arn = event.get("methodArn", "")

    if subject_dn in TRUSTED_CA_DNS:
        return _policy("Allow", method_arn, subject_dn)

    return _policy("Deny", method_arn, subject_dn)

def _policy(effect, method_arn, principal_id):
    return {
        "principalId": principal_id,
        "policyDocument": {
            "Version" : "2012-10-17",
            "Statement": [
                {
                    "Action" : "execute-api:Invoke",
                    "Effect" : effect,
                    "Resource": method_arn,
                }
            ],
        },
    }

```

To add trusted CA DNSs — set the `TRUSTED_CA_DNS` Lambda environment variable to a comma-separated list:

```
CN=My Corp Issuing CA,O=My Corp,C=US,CN=Partner CA,O=Partner Inc,C=GB
```

IP Range Updater (`ip_range_updater.py`)

Triggered by SNS when AWS publishes updated IP ranges. Fetches `ip-ranges.json`, filters for the `AMAZON` service in the configured region, and updates the WAF IP set.

```

import boto3
import json
import os
import urllib.request

wafv2 = boto3.client("wafv2")

IP_SET_ID    = os.environ["WAF_IP_SET_ID"]
IP_SET_NAME  = os.environ["WAF_IP_SET_NAME"]
IP_SET_SCOPE = os.environ.get("WAF_IP_SET_SCOPE", "REGIONAL")
REGION      = os.environ.get("AWS_REGION_FILTER", "us-east-1")
IP_RANGES_URL = "https://ip-ranges.amazonaws.com/ip-ranges.json"

def lambda_handler(event, context):
    with urllib.request.urlopen(IP_RANGES_URL) as resp:
        ip_data = json.loads(resp.read())

    new_ranges = [
        p["ip_prefix"]
        for p in ip_data.get("prefixes", [])
        if p.get("service") == "AMAZON" and p.get("region") == REGION
    ]

    if not new_ranges:
        print(f"No AMAZON ranges found for region {REGION} – skipping update.")
        return

    # WAF UpdateIPSet requires the current lock token
    current = wafv2.get_ip_set(
        Id      = IP_SET_ID,
        Name    = IP_SET_NAME,
        Scope   = IP_SET_SCOPE,
    )

    wafv2.update_ip_set(
        Id          = IP_SET_ID,
        Name        = IP_SET_NAME,
        Scope       = IP_SET_SCOPE,
        LockToken   = current["LockToken"],
        Addresses   = new_ranges,
    )

    print(f"Updated WAF IP set with {len(new_ranges)} AMAZON ranges for {REGION}.")

```

Lambda build script ([build.sh](#))

```
#!/usr/bin/env bash
set -euo pipefail

LAMBDA_DIR="$(cd "$(dirname "$0")" && pwd)"
OUT_ZIP="${LAMBDA_DIR}/token_proxy.zip"

echo "-> Packaging Lambda functions..."
cd "$LAMBDA_DIR"
zip -j "$OUT_ZIP" token_proxy.py authorizer.py ip_range_updater.py
echo "✓ Created ${OUT_ZIP}"
```

Run with:

```
chmod +x lambda/build.sh
./lambda/build.sh
```

8. WAF Rules Reference

Rule 1 — AWS Managed Common Rule Set (Priority 1)

Type: Managed rule group

Action: Count (size restrictions) / Block (all others)

Purpose: Baseline OWASP protections — SQL injection, cross-site scripting, malformed input.

`SizeRestrictions_BODY` is overridden to Count because Cognito token POST bodies are always small and this rule would otherwise generate false positives.

Rule 2 — Block Non-Token Paths (Priority 2)

Type: Byte match (NOT statement)

Action: Block

Purpose: The WAF Web ACL protects the entire Cognito User Pool hosted domain. This rule restricts processing to the `/oauth2/token` path only, reducing the attack surface.

```
NOT (uri_path STARTS_WITH "/oauth2/token" [lowercase]) → BLOCK
```

Rule 3 — Require Internal Secret Header (Priority 3) ← primary gate

Type: Byte match (exact)

Action: Allow

Purpose: The Lambda Token Proxy injects `X-Internal-Token: <64-char-secret>` into every

request to Cognito. The WAF checks for this exact value. Any direct caller — even one with valid Cognito credentials — will be blocked because they cannot know this secret.

```
header[x-internal-token] EXACTLY == <ssm-secret-value> → ALLOW
```

Secret rotation: Update the SSM parameter value, re-deploy the Lambda (picks up new value at next cold start), then update the WAF rule. Use Lambda provisioned concurrency or a canary deployment to avoid gaps.

Rule 4 — Allow AWS AMAZON IP Ranges (Priority 4)

Type: IP set reference

Action: Allow

Purpose: Belt-and-suspenders. Even if the internal secret were somehow leaked, the source IP must still fall within AWS-owned address space. The IP set is automatically refreshed by the IP Range Updater Lambda.

Note: This allows all AMAZON-service IPs in the region, not just API Gateway. It is a secondary control — Rule 3 is the primary gate.

Rule 5 — Rate Limit Token Endpoint (Priority 5)

Type: Rate-based (per IP, 5-minute window)

Action: Block

Limit: 100 requests per 5-minute window per IP (configurable via `var.waf_rate_limit`)

Purpose: Mitigates brute-force and credential-stuffing attacks against the token endpoint.

Default Action — Block 403

Any request that does not match an Allow rule is blocked with:

```
HTTP 403
{
  "error": "access_denied",
  "error_description": "Direct access to this endpoint is not permitted."
}
```

9. Deployment Steps

Step 1 — Build the Lambda package

```
chmod +x lambda/build.sh
./lambda/build.sh
# Output: lambda/token_proxy.zip
```

Step 2 — Configure Terraform variables

```
cd terraform
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars`:

```
aws_region          = "us-east-1"
environment         = "prod"
project             = "oauth-m2m"
cognito_domain_prefix = "my-company-auth"    # must be globally unique in AWS
acm_certificate_arn  = "arn:aws:acm:us-east-1:123456789012:certificate/..."
lambda_zip_path      = "../lambda/token_proxy.zip"
waf_rate_limit       = 100
```

Step 3 — Initialize and apply Terraform

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
```

Step 4 — Upload the mTLS truststore

After `apply` completes, upload your trusted CA certificate chain (PEM format) to the S3 bucket:

```
BUCKET=$(terraform output -raw truststore_bucket)

# Concatenate all trusted CA certs into one PEM file
cat root-ca.pem intermediate-ca.pem > truststore.pem

# Upload with versioning
VERSION=$(aws s3api put-object \
  --bucket "$BUCKET" \
  --key truststore.pem \
  --body truststore.pem \
  --query VersionId --output text)
```



```
echo "S3 version ID: $VERSION"
```

Update `terraform.tfvars` with the version ID and re-apply:

```
truststore_s3_version = "<version-id-from-above>"
```

```
terraform apply
```

Step 5 — Add trusted CA DNs to the Lambda Authorizer

Update the `TRUSTED_CA_DNS` environment variable on the `lambda-authorizer` function with the Distinguished Names of the CA certificates that issued your client certificates:

```
aws lambda update-function-configuration \
  --function-name oauth-m2m-prod-lambda-authorizer \
  --environment "Variables={
    TRUSTED_CA_DNS=CN=My Corp CA\,O=My Corp\,C=US
  }"
```

Or update in `terraform/lambda.tf` under the `TRUSTED_CA_DNS` environment variable block and re-apply.

Step 6 — Configure DNS

Point your custom domain to the API Gateway regional endpoint:

```
terraform output api_gateway_invoke_url
# → https://token.my-company-auth.example.com/token

# Create a CNAME in your DNS provider:
# token.my-company-auth.example.com CNAME <regional-domain-name>.execute-
api.us-east-1.amazonaws.com
```

10. Post-Deployment Verification

Test 1 — Direct call to Cognito (should be blocked by WAF)

```
COGNITO_DOMAIN="https://my-company-auth.auth.us-east-1.amazoncognito.com"
CLIENT_ID="<your-client-id>"
CLIENT_SECRET="<your-client-secret>"
```

```
curl -s -o /dev/null -w "%{http_code}" \
  -X POST "${COGNITO_DOMAIN}/oauth2/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d
"grant_type=client_credentials&client_id=${CLIENT_ID}&client_secret=${CLIENT_SECRET}"

# Expected: 403
```

Test 2 — Call through API Gateway without a client certificate (should be rejected)

```
API_URL="https://token.my-company-auth.example.com/token"

curl -s -o /dev/null -w "%{http_code}" \
  -X POST "${API_URL}" \
  -H "Authorization: Basic $(echo -n "${CLIENT_ID}:${CLIENT_SECRET}" |
base64)" \
  -H "Content-Type: application/x-www-form-urlencoded"

# Expected: 403 (no client cert - mTLS handshake fails or authorizer
denies)
```

Test 3 — Successful call with valid client certificate

```
API_URL="https://token.my-company-auth.example.com/token"

curl -s \
  --cert ./client.crt \
  --key ./client.key \
  --cacert ./ca.pem \
  -X POST "${API_URL}" \
  -H "Authorization: Basic $(echo -n "${CLIENT_ID}:${CLIENT_SECRET}" |
base64)" \
  -H "Content-Type: application/x-www-form-urlencoded"

# Expected: 200 with JSON body:
# {
#   "access_token": "eyJ...",
#   "token_type": "Bearer",
#   "expires_in": 3600
# }
```

Test 4 — Verify WAF logs in CloudWatch

```
aws logs filter-log-events \
  --log-group-name "aws-waf-logs-oauth-m2m-prod-cognito" \
  --filter-pattern "{ $.action = \"BLOCK\" }" \
  --start-time $(date -d '1 hour ago' +%s000) \
  --query 'events[*].message' \
  --output text | head -20
```

11. Operational Notes

Secret rotation

The WAF internal header secret should be rotated periodically:

1. Generate a new secret and update the SSM parameter:

```
NEW_SECRET=$(openssl rand -hex 32)
aws ssm put-parameter \
  --name "/oauth-m2m-prod/waf/internal-header-secret" \
  --value "$NEW_SECRET" \
  --type SecureString \
  --overwrite
```

2. Force a Lambda cold start (update a dummy environment variable or publish a new version).
3. WAF Rule 3 still uses the Terraform-managed value — run **terraform apply** to update it.

Note: There is a brief gap between Lambda cold start and WAF rule update. Use a canary deployment or blue/green Lambda aliases to avoid dropping requests during rotation.

Truststore updates

When a new CA certificate needs to be trusted (e.g., onboarding a new partner):

1. Append the new CA's PEM to **truststore.pem** and re-upload to S3.
2. Note the new S3 version ID.
3. Update **truststore_s3_version** in **terraform.tfvars** and re-apply.
4. Add the new CA's DN to the **TRUSTED_CA_DNS** Lambda environment variable.

IP range refresh

The IP Range Updater Lambda is automatically triggered via SNS whenever AWS publishes updated IP ranges (the **AmazonIpSpaceChanged** topic). No manual intervention is required. You can also trigger it manually:

```
aws lambda invoke \
  --function-name oauth-m2m-prod-ip-range-updater \
```

```
--payload '{} ' \
response.json && cat response.json
```

Monitoring and alerting

Recommended CloudWatch alarms:

Alarm	Metric	Threshold	Action
WAF blocks spike	<code>BlockNon0AuthPath</code> or <code>RequireInternalHeader</code>	> 50 blocks / 5 min	SNS → PagerDuty
Rate limit triggered	<code>RateLimitToken</code>	> 1 block / 5 min	SNS alert
Authorizer deny rate	Lambda authorizer error rate	> 5%	SNS alert
Cognito token failures	Lambda proxy 4xx rate	> 10%	SNS alert

Cost considerations

Resource	Billing model
API Gateway	\$3.50 per million API calls + data transfer
Lambda	Free tier: 1M requests / month; then \$0.20 per million
WAF Web ACL	\$5/month per ACL + \$1 per million requests
WAF rules	\$1/month per rule
Cognito User Pool	Free up to 50,000 MAUs for M2M
CloudWatch Logs	\$0.50 per GB ingested

Last updated: 2026-06-12